20261007: Log-Scaled CMA-ES Parameterization in `glike/estimate.py`

Summary

I refactored / restructured / re-implemented the CMA-ES method in `glike/estimate.py` such that we now employ a single internally consistent log-scale transformation for parameter space between 0-1, based on defined boundary conditions. This replaces the prior scheme utilizing a provided internal scaling / normalization factor.

The conceptual change is:

$$\boxed{\text{old:} \quad q = \frac{x}{s}}$$

to

$$\boxed{\text{new:} \quad u = \text{Normalize}\left[\log(x - L)\right].}$$

The inverse transformation reconstructs the original parameter through

$$\boxed{x = L + \exp\left(z_L + u(z_U - z_L)\right).}$$

Motivation

The original CMA-ES implementation in `estimate.py` attempted to make parameters numerically comparable by assigning characteristic parameter scales. For example, population-size parameters beginning with `N_` were assigned a scale of `50,000`, while parameters beginning with `r`, `m`, or `gr` were assigned a scale of `1.0`.

This approach made the numerical magnitude of the parameters more comparable, but it did not provide a uniform interpretation of the CMA-ES parameter search space.

In particular, a fixed scale such as `N / 50000` does not account for the actual lower and upper bounds of the parameter. A value of `0.05` in normalized space could therefore represent very different changes in different parameters. This is particularly true considering that normalization factors were

user-provided or hard-coded. In the case that factors were not provided, then they were automated; however, this could cause substantial issues for time parameters, which scale in increasing value.

The revised implementation instead makes the CMA-ES log-scaled coordinate itself the fundamental optimization variable. Each parameter is now mapped into a common normalized interval of [0, 1], while the underlying parameter is represented on a logarithmic scale.

The goals are:

1. Put all CMA-ES parameters into a common logarithmic scale.
2. Give `precision=0.05` a consistent interpretation across parameters.
3. Treat supplied parameter bounds as actual hard constraints. Note: Prior implementations were unable to apply boundaries during internal iterations of the CMA-ES optimizer
4. Preserve strict ordering of demographic time parameters.
5. Eliminate model-specific parameter clipping and repair from the CMA-ES objective.
6. Use one mathematical transformation rather than mixing linear scaling, arbitrary parameter scales, and special-case clipping. The prior implementation was several solutions pasted together.

Previous CMA-ES parameterization

The original implementation followed three conceptual steps:

```
original parameter --> time encoding, where applicable --> division by parameter-specific scale
```

The characteristic scale was determined approximately as:

```
if name.startswith("N_"):
    scale = 50000
elif name.startswith("r"):
    scale = 1.0
elif name.startswith("m"):
    scale = 1.0
elif name.startswith("gr"):
    scale = 1.0
```

For parameters without an explicitly assigned scale, the finite bound range could be used as a fallback.

Thus, the normalization was essentially

$$q = \frac{x}{s},$$

where s was a parameter-specific scale.

This has an important limitation: the numerical value of q does not itself represent a bounded coordinate, it was interpreted relative to s. Depending on the relationship between x and s, q could be below, inside, or above the interval $[0, 1]$.

For example, if a parameter had

$$x = 100000, \qquad s = 50000,$$

then

$$q = 2.$$

Consequently, the scale was a numerical convenience rather than a mathematically defined coordinate system.

Revised parameterization

The revised implementation makes the normalized CMA-ES coordinate

$$u \in [0, 1]$$

the fundamental internal parameter representation **for each parameter, even those not necessarily bounded**.

The transformation is based on a logarithmic parameterization.

For a positive parameter x with lower bound L, define its distance from the lower bound as

$$d = x - L.$$

The logarithmic coordinate is then

$$z = \log(d).$$

This transformation changes the interpretation of distance.

In ordinary parameter space, a difference of

$$x_2 - x_1$$

is additive.

In log space, a difference of

$$\log(x_2 - L) - \log(x_1 - L)$$

corresponds to a multiplicative ratio:

$$\log\left(\frac{x_2 - L}{x_1 - L}\right).$$

Thus equal distances in log space correspond to equal multiplicative changes in the parameter's distance from its lower bound.

Mapping log space to the CMA-ES interval

For a parameter with finite bounds

$$L < x < U,$$

the log-space endpoints are

$$z_L = \log(x_{\min} - L)$$

and

$$z_U = \log(U - L),$$

where the lower endpoint must be represented by a strictly positive distance because

$$\log(0)$$

is undefined.

The normalized CMA-ES coordinate is obtained by linearly mapping the log-space coordinate into $[0, 1]$:

$$u = \frac{z - z_L}{z_U - z_L}.$$

Equivalently,

$$z = z_L + u(z_U - z_L).$$

The original parameter is recovered using the exponential:

$$x = L + \exp(z).$$

Combining these equations gives the direct transformation

$$\boxed{x(u) = L + \exp\left[z_L + u(z_U - z_L)\right]}$$

for a finite bounded positive parameter.

The inverse transformation is

$$\boxed{u(x) = \frac{\log(x - L) - z_L}{z_U - z_L}}.$$

Therefore:

$$u = 0$$

corresponds to the lower endpoint of the allowed log-space interval, while

$$u = 1$$

corresponds to the upper endpoint.

Importantly, `u = 1` does **not** mean that the demographic parameter itself has value `1`. **It means that the optimizer is at the upper end of the parameter's permitted log-scaled range.**

Rational: Logarithmic transformation

Many demographic parameters have strictly positive domains and can span orders of magnitude. Population sizes, for example, can plausibly differ by factors rather than by fixed additive increments. Suppose two candidate values are

$$x_1 = 1000$$

and

$$x_2 = 2000.$$

Their difference is 1000, but the second value is also exactly twice the first.

Likewise,

$$x_3 = 10000$$

and

$$x_4 = 20000$$

also differ by 10000 but represent the same multiplicative change:

$$\frac{x_4}{x_3} = \frac{x_2}{x_1} = 2.$$

In log space,

$$\log(2000) - \log(1000) = \log(2)$$

and

$$\log(20000) - \log(10000) = \log(2).$$

Therefore, equal movement in the transformed space represents equal fold-change in the original parameter.

This is desirable for parameters whose relevant scale is multiplicative, as is true for much of biology.

Ordered demographic times

Demographic models frequently require strict temporal ordering:

$$t_1 < t_2 < t_3 < t_4.$$

Directly optimizing the four times independently does not guarantee this condition. The implementation therefore retains the interval representation of ordered times.

Define positive intervals

$$d_1 = t_1$$

$$d_2 = t_2 - t_1$$

$$d_3 = t_3 - t_2$$

$$d_4 = t_4 - t_3.$$

The times are reconstructed as

$$t_1 = d_1$$

$$t_2 = d_1 + d_2$$

$$t_3 = d_1 + d_2 + d_3$$

$$t_4 = d_1 + d_2 + d_3 + d_4.$$

Each interval is positive, so the ordering follows automatically:

$$t_1 < t_2 < t_3 < t_4.$$

The intervals themselves are subjected to the same logarithmic parameterization used for other positive quantities.

This is preferable to generating arbitrary times and subsequently repairing them with operations such as

```
t2 = t1 + 1e-5
```

because the ordering constraint is built into the parameterization itself.

Removal of ad-hoc parameter repair

The previous CMA-ES objective contained model-specific validity corrections.

Examples included:

```
if params["N_anc"] <= 0:
    params["N_anc"] = 1e-5
```

and:

```
if params["t2"] <= params["t1"]:
    params["t2"] = params["t1"] + 1e-5
```

Growth rates were similarly clipped to an interval close to `[0, 1]`. These operations effectively changed a candidate proposed by CMA-ES before evaluating it. This creates an undesirable distinction between:

- the point proposed by the optimizer, and
- the point actually evaluated by the likelihood function.

The revised implementation instead makes invalid points genuinely invalid.

The intended flow is now:

```
CMA-ES candidate --> normalized coordinate --> inverse log transformation --> original parameter
```

If a candidate does not correspond to a valid demographic parameterization, its objective value is returned as

```
np.inf
```

because CMA-ES is minimizing the negative likelihood.

No parameter is silently moved to a nearby valid value. This was causing some issues in prior implementations

Treatment of parameter bounds

Bounds are now treated as part of the mathematical parameterization rather than merely as a post-hoc validity check. For a finite bounded parameter, the lower and upper bounds determine the corresponding endpoints of the transformed coordinate. Conceptually:

```
Lower   <-->    Upper
        becomes
z_Lower <-->    Z_Upper
        normalizes
0       <-->    1
```

This means that CMA-ES always operates in the same normalized domain regardless of whether the original parameter is measured in generations, population size, migration rate, or another demographic unit.

Fringe case: Infinite upper bounds

A fully logarithmic transformation of an unbounded positive parameter presents a mathematical issue.

For

$$x \in (L, \infty),$$

the logarithmic coordinate has no finite upper endpoint:

$$\log(x - L) \to \infty.$$

Consequently, there is no natural affine mapping of the entire positive half-line onto a finite interval `[0,1]`.

The implementation therefore uses the supplied `parameter_scales` only where an infinite upper bound prevents the construction of a finite transformed interval. In that situation, the scale supplies a characteristic reference distance in transformed space.The important conceptual distinction is that the scale is **not** an alternative normalization system. It only supplies the missing finite reference needed to parameterize an otherwise unbounded logarithmic domain. For parameters with finite bounds, the actual bounds take precedence.

Note: Interpretation of `precision` in CMA-ES

After the transformation, CMA-ES is initialized in normalized space.

Therefore:

```
precision = 0.05
```

means that the initial CMA-ES search distribution has a standard deviation of approximately

$$0.05$$

normalized units.

The important consequence is that `precision` no longer directly corresponds to a fixed additive change in the original parameter. Because the transformation is logarithmic, a movement in normalized space corresponds to a multiplicative change in the original parameter.

Thus the same CMA-ES step size can produce:

- a relatively small additive change for one parameter,
- a substantially larger additive change for another parameter,

while representing comparable movement in their respective logarithmic scales.

New optimization workflow

The resulting workflow is:

1. Original demographic parameters (input initial conditions)
2. Ordered-time encoding (dt_1, dt_2, dt_3, dt_4, etc.)
3. Log transformation
4. Normalize to u in [0, 1]
5. Apply CMA-ES
6. Yield Normalized candidate
7. Inverse of normalization of candidate solution
8. Exponential transform to inverse log-scaling
9. Reconstruct ordered times (t_1, t_2, t_3, t_4, etc.)
10. Hard-bound validation
11. Likelihood evaluation

Mathematical rationale

The central reason for this change is that the optimization problem is better represented as

$$\boxed{\max_{x \in \mathcal{X}} f(x)}$$

with a transformation

$$x = T(u), \qquad u \in [0,1]^n.$$

CMA-ES therefore operates on

$$\boxed{\max_{u \in [0,1]^n} f(T(u))}$$

or equivalently

$$\boxed{\min_{u \in [0,1]^n} -f(T(u)).}$$

The revised implementation should provide several advantages:

- Consistent CMA-ES coordinate system
    - Every parameter is represented in the same normalized domain:

$$u \in [0, 1].$$

- Multiplicative parameter geometry
  - Equal movement in transformed space corresponds to comparable fold changes rather than comparable absolute changes.
- Explicit constraints
  - Parameter bounds are incorporated into the transformation instead of being repaired after sampling.
- Guaranteed time ordering
  - Ordered demographic events are represented through positive intervals, making the ordering structural rather than corrective.
- No silent clipping at soft parameter boundaries
  - An invalid candidate is rejected rather than changed into a different candidate.
- More interpretable `precision`
  - The CMA-ES initial step size is now defined in a common normalized space.

Limitations

Infinite values

A finite `[0,1]` representation and a genuinely unbounded positive logarithmic parameter space cannot both be obtained without introducing some additional scale or saturation convention.

Mathematically,

$$\log(x - L)$$

extends to infinity as

$$x \to \infty.$$

Therefore, if a parameter has no finite upper bound, some finite reference scale or alternative bounded transformation is necessarily required if CMA-ES is to remain in `[0,1]`.

This is why finite bounds are preferred whenever scientifically justified.

The revised implementation consequently treats explicit finite bounds as the most principled definition of the CMA-ES search domain.

Negative values

A seperate issue arises in which logarithmic scales cannot apply to negative values. Almost all parameters have some element of negative value; however, the growth parameters **can be negative**, e.g., in the case of population decline. As currently implemented, because all parameters undergo a logscaling transformation, parameters which may natively possess a negative value are not fit as well.